

MultiOmicsSurvey Benchmark Function Documentation

MultiOmicsSurvey Project

2026-01-15

Contents

1	Introduction	3
2	Utility Scripts (performance_jobs/)	4
2.1	Create_feature_ranks.R	4
2.1.1	rank_mad_ids	4
2.2	Create_sample_subsets.R	4
2.2.1	strat_sample_ids	4
3	Algorithm Benchmark Scripts	6
3.1	ab-SNF	6
3.1.1	get_top_features	6
3.1.2	subset_input_features	6
3.1.3	subset_input_cols	7
3.1.4	mad_based_weights	7
3.1.5	weighted_euclidean_dist	8
3.1.6	weighted_hamming_fraction_fast	8
3.1.7	spectralClustering_eig	9
3.1.8	ari_to_ground_truth	10
3.1.9	get_int1	10
3.2	ANF	11
3.3	CIMLR	12
3.3.1	extract_m3c_scores	12
3.3.2	extract_m3c_clusters	12
3.4	COCA	14
3.4.1	coca_distances_for	14
3.4.2	to_samples_x_features	14
3.4.3	align_by_samples	14

3.5	iClusterBayes	16
3.5.1	with_install_lock	16
3.5.2	ensure_cran_pkg	16
3.5.3	ensure_bioc_pkg	17
3.5.4	align_modalities	17
4	KLIC	18
4.1	ensure_mosek_env	18
4.2	ensure_rmosek	18
4.3	to_sample_feature	19
4.4	align_by_common_samples	19
5	LRACluster	20
5.1	types_by_mod	20
6	MDICC	21
6.1	get_mdicc_field	21
6.2	same_conda_prefix	21
6.3	ensure_python	21
6.4	ensure_projsplx	22
6.5	fallback_testaff	22
6.6	fallback_MDICClablel	23
7	MONET	24
7.1	parse_args (MONET R)	24
7.2	align_common_samples (MONET)	24
8	MSNE	25
8.1	parse_timev	25
9	NEMO	26
10	RWR-F	27
10.1	parse_args (RWR-F)	27
10.2	specc_with_embedding	27
11	SNF	28
12	Spectrum	29
12.1	ref_modality	29
12.2	safe_silhouette_avg	29

13 wMKL	30
13.1 weighted_mutations	30
13.2 extract_eigenspace	30
13.3 safe_avg_sil	31
14 Python Scripts	32
14.1 MONET (run_MONET_mainloop.py)	32
14.1.1 set_seeds (Python)	32
14.1.2 load_correlation_matrices	32
14.2 MSNE (MSNE_run.py)	32
14.2.1 set_seeds (MSNE Python)	32
14.2.2 load_views_from_csv	33
15 Post-Analysis Scripts (automated_scripts/)	34
15.1 benchmark_plotting_functions.R	34
15.1.1 theme_benchmark	34
15.1.2 guess_benchmark_dirs	34
15.1.3 read_mode_perf	35
15.1.4 plot_metric_lines	35
15.1.5 save_plot_png_pdf	36
15.1.6 fit_time_scaling_models	36
15.1.7 compute_sample_stability_90pct	37
15.1.8 make_benchmark_plots	37
16 Alphabetical Index	38

1 Introduction

This document provides comprehensive documentation for all custom functions used in the performance benchmarking scripts of the MultiOmicsSurvey project. Functions are organized by source file location.

2 Utility Scripts (performance_jobs/)

2.1 Create_feature_ranks.R

Script for creating feature rankings based on variance metrics for benchmarking perturbation experiments.

2.1.1 rank_mad_ids

```
rank_mad_ids <- function(mat) {
  stopifnot(!is.null(rownames(mat)))
  m <- apply(mat, 1, function(x) mad(x, na.rm = TRUE))
  keep <- which(!is.na(m))
  ord <- keep[order(m[keep], decreasing = TRUE)]
  rownames(mat)[ord]
}
```

Description: Ranks features (rows) of a matrix by their Median Absolute Deviation (MAD) in descending order. Used for continuous modalities (RNAseq, CNV, Methylation, miRNA).

Parameter	Type	Description
mat	matrix	Feature-by-sample matrix with rownames as feature IDs

Returns: Character vector of feature IDs ordered by decreasing MAD.

2.2 Create_sample_subsets.R

Script for creating stratified sample subsets for benchmarking sample perturbation experiments.

2.2.1 strat_sample_ids

```
strat_sample_ids <- function(frac, B, er_factor_named) {
  ids_all <- names(er_factor_named)
  levs <- levels(er_factor_named)
  out <- vector("list", B)

  for (b in seq_len(B)) {
    picked <- character(0)
    for (lv in levs) {
      ids_lv <- ids_all[er_factor_named == lv]
      n_lv <- max(1L, round(length(ids_lv) * frac / 100))
      n_lv <- min(n_lv, length(ids_lv))
      picked <- c(picked, sample(ids_lv, size = n_lv, replace = FALSE))
    }
    out[[b]] <- picked
  }
}
```

```
}  
  out  
}
```

Description: Creates stratified random samples of sample IDs, maintaining the proportion of each stratum (e.g., ER status). Generates B replicate subsets at a given fraction.

Parameter	Type	Description
<code>frac</code>	numeric	Percentage of samples to include (0-100)
<code>B</code>	integer	Number of replicate subsets to generate
<code>er_factor_named</code>	factor	Named factor with sample IDs as names and strata as levels

Returns: List of length B, each element containing a character vector of sampled IDs.

3 Algorithm Benchmark Scripts

3.1 ab-SNF

Source files: ab-SNF_feature_experiments.R, ab-SNF_sample_experiments.R

3.1.1 get_top_features

```
get_top_features <- function(feet_rank_dt, centile) {
  mods <- unique(feet_rank_dt$modality)
  out <- vector("list", length(mods)); names(out) <- mods
  for (mod in mods) {
    mod_dt <- feet_rank_dt[modality == mod][order(rank)]
    top_n <- max(1L, ceiling(centile * nrow(mod_dt)))
    out[[mod]] <- mod_dt$feature_id[seq_len(top_n)]
  }
  out
}
```

Description: Extracts the top-ranked features for each modality based on a centile threshold.

Parameter	Type	Description
feat_rank_dt	data.table	Feature rankings with columns: modality, rank, feature_id
centile	numeric	Fraction of features to select (0-1)

Returns: Named list of character vectors, one per modality, containing selected feature IDs.

3.1.2 subset_input_features

```
subset_input_features <- function(input_obj, top_features) {
  out <- input_obj
  for (mod in names(top_features)) {
    if (!is.null(out[[mod]])) {
      keep <- intersect(top_features[[mod]], rownames(out[[mod]]))
      out[[mod]] <- out[[mod]][keep, , drop = FALSE]
      if (nrow(out[[mod]]) == 0L) out[[mod]] <- NULL
    }
  }
  out
}
```

Description: Subsets an input object (list of matrices) to retain only specified features per modality.

Parameter	Type	Description
input_obj	list	Named list of feature-by-sample matrices
top_features	list	Named list of feature IDs to retain per modality

Returns: Modified input object with subsetted matrices.

3.1.3 subset_input_cols

```
subset_input_cols <- function(input_obj, keep_ids) {
  out <- input_obj
  for (nm in names(out)) {
    if (!is.null(out[[nm]])) {
      kk <- keep_ids[keep_ids %in% colnames(out[[nm]])]
      out[[nm]] <- out[[nm]][, kk, drop = FALSE]
      if (ncol(out[[nm]]) == 0L) out[[nm]] <- NULL
    }
  }
  out
}
```

Description: Subsets an input object (list of matrices) to retain only specified samples (columns).

Parameter	Type	Description
input_obj	list	Named list of feature-by-sample matrices
keep_ids	character	Sample IDs to retain

Returns: Modified input object with subsetted columns.

3.1.4 mad_based_weights

```
mad_based_weights <- function(X) {
  v <- apply(X, 2, mad, na.rm = TRUE)
  v[!is.finite(v)] <- 0
  s <- sum(v)
  if (s <= 0) rep(0, length(v)) else v / s
}
```

Description: Computes feature weights based on column-wise MAD values, normalized to sum to 1.

Parameter	Type	Description
X	matrix	Sample-by-feature matrix

Returns: Numeric vector of normalized weights.

3.1.5 weighted_euclidean_dist

```
weighted_euclidean_dist <- function(X, w, ids) {
  Xw <- sweep(X, 2, sqrt(w), `*`)
  d <- abSNF::dist2(as.matrix(Xw), as.matrix(Xw))
  dimnames(d) <- list(ids, ids)
  d
}
```

Description: Computes weighted Euclidean distance matrix using feature weights.

Parameter	Type	Description
X	matrix	Sample-by-feature data matrix
w	numeric	Feature weights
ids	character	Sample identifiers for row/column names

Returns: Square distance matrix with sample IDs as dimnames.

3.1.6 weighted_hamming_fraction_fast

```
weighted_hamming_fraction_fast <- function(Xbin, features, drivers,
                                           driver_w = 0.8, nondriver_w = 0.2, ids) {
  X <- (as.matrix(Xbin) != 0) * 1
  w <- ifelse(features %in% drivers, driver_w, nondriver_w)
  w[!is.finite(w)] <- 0
  total_w <- sum(w)
  if (total_w <= 0) {
    d <- matrix(0, nrow = nrow(X), ncol = nrow(X))
    dimnames(d) <- list(ids, ids)
    return(d)
  }
  Xw <- sweep(X, 2, sqrt(w), `*`)
  C <- Xw %*% t(Xw)
  s <- as.numeric(X %*% w)
  d <- outer(s, s, "+") - 2 * C
  d <- pmax(d, 0)
  d <- d / total_w
  dimnames(d) <- list(ids, ids)
  d
}
```


Description: Computes weighted Hamming distance for binary data, with higher weights for driver genes.

Parameter	Type	Description
Xbin	matrix	Binary sample-by-feature matrix
features	character	Feature names (column names)
drivers	character	Driver gene names to upweight
driver_w	numeric	Weight for driver features (default: 0.8)
nondriver_w	numeric	Weight for non-driver features (default: 0.2)
ids	character	Sample identifiers

Returns: Square distance matrix with sample IDs as dimnames.

3.1.7 spectralClustering_eig

```
spectralClustering_eig <- function(affinity, K, type = 3) {
  ids <- colnames(affinity) %||% rownames(affinity) %||% paste0("S", seq_len(nrow(affinity)))

  d <- rowSums(affinity)
  d[d == 0] <- .Machine$double.eps
  D <- diag(d)
  L <- D - affinity
  if (type == 1) {
    NL <- L
  } else if (type == 2) {
    NL <- diag(1 / d) %*% L
  } else {
    Di <- diag(1 / sqrt(d))
    NL <- Di %*% L %*% Di
  }

  eig <- eigen(NL)
  res <- sort(abs(eig$values), index.return = TRUE)
  U <- eig$vectors[, res$ix[1:K], drop = FALSE]

  if (type == 3) {
    normalize <- function(x) x / sqrt(sum(x^2))
    U <- t(apply(U, 1, normalize))
  }

  eigDiscrete <- SNFtool:::.discretisation(U)$discrete
  labels <- apply(eigDiscrete, 1, which.max)

  out <- as.data.frame(U)
  out$Cluster <- labels
  out$Sample.ID <- ids
  out
}
```

Description: Performs spectral clustering on an affinity matrix using eigendecomposition of the graph Laplacian.

Parameter	Type	Description
affinity	matrix	Square affinity/similarity matrix
K	integer	Number of clusters
type	integer	Laplacian type: 1=unnormalized, 2=random walk, 3=symmetric (default)

Returns: Data frame with eigenvector embeddings, cluster assignments, and sample IDs.

3.1.8 ari_to_ground_truth

```
ari_to_ground_truth <- function(gt_df, pred_df) {
  merged <- merge(gt_df, pred_df, by = "Sample.ID")
  if (nrow(merged) == 0) return(NA_real_)
  adjustedRandIndex(merged[["Cluster_gt"]], merged[["Cluster_pred"]])
}
```

Description: Computes Adjusted Rand Index between predicted clusters and ground truth labels.

Parameter	Type	Description
gt_df	data.frame	Ground truth with columns: Sample.ID, Cluster_gt
pred_df	data.frame	Predictions with columns: Sample.ID, Cluster_pred

Returns: Numeric ARI value, or NA if no samples match.

3.1.9 get_int1

```
get_int1 <- function(x) {
  if (is.null(x) || length(x) == 0L || is.na(x)) NA_integer_ else as.integer(x[1])
}
```

Description: Safely extracts the first element as integer, returning NA for NULL/empty/NA inputs.

Parameter	Type	Description
x	any	Input value

Returns: Integer or NA_integer_.

3.2 ANF

Source files: `ANF_feature_experiments.R`, `ANF_sample_experiments.R`

Note: ANF scripts share many functions with ab-SNF (`get_top_features`, `subset_input_features`, `spectralClustering_eig`, `ari_to_ground_truth`). The ANF scripts use the same helper functions but call `ANF::affinity_matrix()` and `ANF::ANF()` for network fusion instead of SNF.

3.3 CIMLR

Source files: CIMLR_feature_experiments.R, CIMLR_sample_experiments.R

Note: CIMLR scripts share get_top_features, subset_input_features, subset_input_cols, ari_to_ground_truth with other algorithms.

3.3.1 extract_m3c_scores

```
extract_m3c_scores <- function(m3c_obj, k = 2L) {
  sc <- m3c_obj$scores
  if (is.null(sc)) return(list(p = NA_real_, entropy = NA_real_, rcsi = NA_real_))
  sc <- as.data.frame(sc)
  row <- if ("K" %in% colnames(sc)) sc[sc$K == k, , drop = FALSE] else sc[k - 1L, , drop = FALSE]
  if (nrow(row) == 0) row <- sc[1, , drop = FALSE]

  p_col <- if ("NORM_P" %in% colnames(row)) "NORM_P"
    else if ("MONTECARLO_P" %in% colnames(row)) "MONTECARLO_P"
    else if ("BETA_P" %in% colnames(row)) "BETA_P" else NA_character_
  e_col <- if ("Entropy" %in% colnames(row)) "Entropy" else {
    cand <- grep("entropy", colnames(row), ignore.case = TRUE, value = TRUE)
    if (length(cand)) cand[1] else NA_character_
  }
  r_col <- if ("RCSI" %in% colnames(row)) "RCSI" else {
    cand <- grep("^RCSI", colnames(row), value = TRUE)
    if (length(cand)) cand[1] else NA_character_
  }

  list(
    p = if (!is.na(p_col)) as.numeric(row[[p_col]][1]) else NA_real_,
    entropy = if (!is.na(e_col)) as.numeric(row[[e_col]][1]) else NA_real_,
    rcsi = if (!is.na(r_col)) as.numeric(row[[r_col]][1]) else NA_real_
  )
}
```

Description: Extracts M3C consensus clustering scores for a specified number of clusters.

Parameter	Type	Description
m3c_obj	list	M3C result object
k	integer	Number of clusters to extract scores for

Returns: Named list with p-value, entropy, and RCSI scores.

3.3.2 extract_m3c_clusters

```

extract_m3c_clusters <- function(m3c_obj, k = 2L) {
  if (!is.null(m3c_obj$realdataresults) && length(m3c_obj$realdataresults) >= k) {
    ann <- m3c_obj$realdataresults[[k]]$ordered_annotation
    if (!is.null(ann) && "consensuscluster" %in% colnames(ann)) {
      ids <- if ("ID" %in% colnames(ann)) ann$ID else rownames(ann)
      return(data.frame(Sample.ID = ids, Cluster_pred = as.integer(ann$consensuscluster)))
    }
  }
  NULL
}

```

Description: Extracts consensus cluster assignments from an M3C result object.

Parameter	Type	Description
m3c_obj	list	M3C result object
k	integer	Number of clusters

Returns: Data frame with Sample.ID and Cluster_pred columns, or NULL if unavailable.

3.4 COCA

Source files: COCA_feature_experiments.R, COCA_sample_experiments.R

Note: COCA scripts share get_top_features, subset_input_features, subset_input_cols, ari_to_ground_truth with other algorithms.

3.4.1 coca_distances_for

```
coca_distances_for <- function(mod_names) {
  vapply(mod_names, function(m) if (identical(m, "SNPs")) "binary" else "euclidean", character(1))
}
```

Description: Assigns appropriate distance metric for each modality (binary for SNPs, euclidean for others).

Parameter	Type	Description
mod_names	character	Vector of modality names

Returns: Named character vector of distance metrics.

3.4.2 to_samples_x_features

```
to_samples_x_features <- function(mat) {
  stopifnot(!is.null(rownames(mat)), !is.null(colnames(mat)))
  t(as.matrix(mat))
}
```

Description: Transposes a feature-by-sample matrix to sample-by-feature format required by COCA.

Parameter	Type	Description
mat	matrix	Feature-by-sample matrix with dimnames

Returns: Transposed matrix (samples as rows, features as columns).

3.4.3 align_by_samples

```
align_by_samples <- function(X_list) {
  ids_list <- lapply(X_list, rownames)
  common <- Reduce(intersect, ids_list)
  if (length(common) == 0) stop("No common sample IDs across modalities after transpose.")
  common <- common[common %in% ids_list[[1]]]
  lapply(X_list, function(m) m[common, , drop = FALSE])
}
```

Description: Aligns a list of sample-by-feature matrices to common samples across all modalities.

Parameter	Type	Description
<code>x_list</code>	list	Named list of sample-by-feature matrices

Returns: List of matrices with identical rownames (samples).

3.5 iClusterBayes

Source files: `iClusterBayes_feature_experiments.R`, `iClusterBayes_sample_experiments.R`

Note: iClusterBayes scripts share `get_top_features`, `subset_input_features`, `subset_input_cols` with other algorithms.

3.5.1 with_install_lock

```
with_install_lock <- function(expr) {
  lockdir <- file.path(user_lib, ".pkg_install_lockdir")
  repeat {
    if (dir.create(lockdir, showWarnings = FALSE)) break
    Sys.sleep(2)
  }
  on.exit(unlink(lockdir, recursive = TRUE, force = TRUE), add = TRUE)
  force(expr)
}
```

Description: Executes an expression with a directory-based lock to prevent concurrent package installations in HPC array jobs.

Parameter	Type	Description
<code>expr</code>	expression	Expression to evaluate under lock

Returns: Result of the evaluated expression.

3.5.2 ensure_cran_pkg

```
ensure_cran_pkg <- function(pkg) {
  if (requireNamespace(pkg, quietly = TRUE)) return(invisible(TRUE))
  with_install_lock({
    install.packages(pkg, lib = user_lib, repos = "https://cran.r-project.org")
  })
  if (!requireNamespace(pkg, quietly = TRUE)) stop("Failed to install CRAN package: ", pkg)
  invisible(TRUE)
}
```

Description: Ensures a CRAN package is installed, using installation locking for HPC safety.

Parameter	Type	Description
<code>pkg</code>	character	Package name

Returns: Invisible TRUE on success; stops on failure.

3.5.3 ensure_bioc_pkg

```
ensure_bioc_pkg <- function(pkg) {
  ensure_cran_pkg("BiocManager")
  if (requireNamespace(pkg, quietly = TRUE)) return(invisible(TRUE))
  with_install_lock({
    BiocManager::install(pkg, lib = user_lib, ask = FALSE, update = FALSE)
  })
  if (!requireNamespace(pkg, quietly = TRUE)) stop("Failed to install Bioconductor package: ", pkg)
  invisible(TRUE)
}
```

Description: Ensures a Bioconductor package is installed, using installation locking for HPC safety.

Parameter	Type	Description
pkg	character	Package name

Returns: Invisible TRUE on success; stops on failure.

3.5.4 align_modalities

```
align_modalities <- function(X_list, ref_mod = NULL) {
  ids_list <- lapply(X_list, colnames)
  common <- Reduce(intersect, ids_list)
  if (length(common) == 0L) stop("No common sample IDs across modalities.")
  if (!is.null(ref_mod) && ref_mod %in% names(X_list)) {
    common <- common[common %in% colnames(X_list[[ref_mod]])]
  }
  lapply(X_list, function(m) m[, common, drop = FALSE])
}
```

Description: Aligns a list of feature-by-sample matrices to common samples across all modalities.

Parameter	Type	Description
X_list	list	Named list of feature-by-sample matrices
ref_mod	character	Optional reference modality for ordering

Returns: List of matrices with identical colnames (samples).

4 KLIC

Source files: `KLIC_feature_experiments.R`, `KLIC_sample_experiments.R`

Note: KLIC scripts share `get_top_features`, `subset_input_features`, `subset_input_cols`, and `ari_to_ground_truth` with other algorithms.

4.1 `ensure_mosek_env`

```
ensure_mosek_env <- function() {
  mosek_path <- Sys.getenv("MOSEK_PATH", "")
  if (mosek_path == "" || !dir.exists(mosek_path)) {
    stop("MOSEK_PATH not set or invalid. Set Sys.setenv(MOSEK_PATH = '/path/to/mosek').")
  }
  lic_dir <- file.path(mosek_path, "mosek")
  if (!file.exists(file.path(lic_dir, "mosek.lic"))) {
    stop("mosek.lic not found in expected location: ", lic_dir)
  }
  Sys.setenv(MOSEKLM_LICENSE_FILE = file.path(lic_dir, "mosek.lic"))
  invisible(TRUE)
}
```

Description: Ensures MOSEK environment is properly configured for KLIC optimization.

Parameter	Type	Description
(none)	-	Function takes no parameters

Returns: Invisible TRUE on success; stops on failure.

4.2 `ensure_rmosek`

```
ensure_rmosek <- function() {
  ensure_mosek_env()
  if (!requireNamespace("Rmosek", quietly = TRUE)) {
    mosek_r_dir <- file.path(Sys.getenv("MOSEK_PATH"), "10.2", "tools", "platform",
                             "linux64x86", "rmosek")
    if (!dir.exists(mosek_r_dir)) stop("Rmosek directory not found: ", mosek_r_dir)
    install.packages(file.path(mosek_r_dir, "Rmosek_10.2.0.tar.gz"),
                     repos = NULL, type = "source")
  }
  invisible(TRUE)
}
```

Description: Ensures the Rmosek package is installed for KLIC's convex optimization.

Parameter	Type	Description
(none)	-	Function takes no parameters

Returns: Invisible TRUE on success; stops on failure.

4.3 to_sample_feature

```
to_sample_feature <- function(X) {
  if (nrow(X) > ncol(X)) return(X) # Already samples × features

  t(X) # Transpose from features × samples
}
```

Description: Ensures matrix is in samples-by-features orientation for KLIC.

Parameter	Type	Description
X	matrix	Input matrix to orient

Returns: Matrix with samples as rows and features as columns.

4.4 align_by_common_samples

```
align_by_common_samples <- function(X_list) {
  ids_list <- lapply(X_list, rownames)
  common <- Reduce(intersect, ids_list)
  if (length(common) == 0L) stop("No common samples.")
  lapply(X_list, function(m) m[common, , drop = FALSE])
}
```

Description: Aligns multiple matrices by common sample IDs (row names).

Parameter	Type	Description
X_list	list	List of samples-by-features matrices

Returns: List of aligned matrices with identical row names.

5 LRAcluster

Source files: LRAcluster_feature_experiments.R, LRAcluster_sample_experiments.R

Note: LRAcluster scripts share get_top_features, subset_input_features, subset_input_cols, ari_to_ground_truth, align_modalities, and extract_m3c_clusters with other algorithms.

5.1 types_by_mod

```
types_by_mod <- function(mod_names) {
  type_map <- c(
    RNAseq = "gaussian",
    CNV = "gaussian",
    Methylation = "gaussian",
    miRNA = "gaussian",
    SNPs = "binary"
  )
  out <- type_map[mod_names]
  if (anyNA(out)) stop("Unknown modality name(s): ", paste(mod_names[is.na(out)], collapse = ", "))
  out
}
```

Description: Maps modality names to LRAcluster data types.

Parameter	Type	Description
mod_names	character	Vector of modality names

Returns: Named character vector of data types (“gaussian” or “binary”).

6 MDICC

Source files: MDICC_feature_experiments.R, MDICC_sample_experiments.R

Note: MDICC scripts share get_top_features, subset_input_features, subset_input_cols, ari_to_ground_truth, and to_sample_feature with other algorithms.

6.1 get_mdicc_field

```
get_mdicc_field <- function(result, field, default = NULL) {
  if (is.null(result) || !is.list(result)) return(default)
  if (field %in% names(result)) return(result[[field]])
  default
}
```

Description: Safely extracts a field from MDICC result objects.

Parameter	Type	Description
result	list	MDICC result object
field	character	Field name to extract
default	any	Default value if field not found

Returns: Extracted field value or default.

6.2 same_conda_prefix

```
same_conda_prefix <- function(a, b) {
  normalizePath(a, winslash = "/", mustWork = FALSE) ==
  normalizePath(b, winslash = "/", mustWork = FALSE)
}
```

Description: Compares two paths to check if they refer to the same conda environment.

Parameter	Type	Description
a	character	First path
b	character	Second path

Returns: Logical TRUE if paths are equivalent.

6.3 ensure_python

```

ensure_python <- function(conda_env = "mdicc_env") {
  if (!reticulate::condaenv_exists(conda_env)) {
    stop("Conda environment not found: ", conda_env)
  }
  current <- tryCatch(reticulate::py_config()$prefix, error = function(e) "")
  target <- reticulate::conda_python(envname = conda_env)
  target_prefix <- dirname(dirname(target))
  if (!same_conda_prefix(current, target_prefix)) {
    reticulate::use_condaenv(conda_env, required = TRUE)
  }
  invisible(TRUE)
}

```

Description: Ensures the correct Python conda environment is active for MDICC.

Parameter	Type	Description
conda_env	character	Conda environment name (default: "mdicc_env")

Returns: Invisible TRUE on success; stops on failure.

6.4 ensure_projsplx

```

ensure_projsplx <- function() {
  py <- reticulate::import("sys")
  if (!reticulate::py_module_available("projsplx")) {
    reticulate::py_install("projsplx", pip = TRUE)
  }
  invisible(TRUE)
}

```

Description: Ensures the projsplx Python package is installed.

Parameter	Type	Description
(none)	-	Function takes no parameters

Returns: Invisible TRUE on success.

6.5 fallback_testaff

```
fallback_testaff <- function(data_list, k = 5L) {
  n <- nrow(data_list[[1]])
  affs <- lapply(data_list, function(X) {
    D <- as.matrix(dist(X))
    exp(-D^2 / (2 * median(D)^2))
  })
  Reduce(`+`, affs) / length(affs)
}
```

Description: Fallback affinity computation when MDICC Python call fails.

Parameter	Type	Description
<code>data_list</code>	list	List of data matrices
<code>k</code>	integer	Number of neighbors (default: 5)

Returns: Averaged affinity matrix.

6.6 fallback_MDICClabel

```
fallback_MDICClabel <- function(affinity, nclust) {
  km <- kmeans(affinity, centers = nclust, nstart = 25L)
  list(labels = km$cluster, converged = TRUE, obj = km$tot.withinss)
}
```

Description: Fallback clustering when MDICC label function fails.

Parameter	Type	Description
<code>affinity</code>	matrix	Affinity matrix
<code>nclust</code>	integer	Number of clusters

Returns: List with labels, convergence flag, and objective value.

7 MONET

Source files: `MONET_make_correlations_feature.R`, `MONET_make_correlations_sample.R`, `run_MONET_mainloop.py`

Note: MONET R scripts share `get_top_features`, `subset_input_features`, and `ari_to_ground_truth` with other algorithms.

7.1 `parse_args` (MONET R)

```
parse_args <- function() {
  args <- commandArgs(trailingOnly = TRUE)
  if (length(args) < 2L) stop("Usage: Rscript script.R <input_dir> <output_dir>")
  list(input_dir = args[1], output_dir = args[2])
}
```

Description: Parses command-line arguments for MONET R scripts.

Parameter	Type	Description
(none)	-	Reads from <code>commandArgs</code>

Returns: List with `input_dir` and `output_dir` paths.

7.2 `align_common_samples` (MONET)

```
align_common_samples <- function(X_list) {
  ids_list <- lapply(X_list, colnames)
  common <- Reduce(intersect, ids_list)
  if (length(common) == 0L) stop("No common samples.")
  lapply(X_list, function(m) m[, common, drop = FALSE])
}
```

Description: Aligns feature-by-sample matrices to common column names.

Parameter	Type	Description
<code>X_list</code>	list	List of feature-by-sample matrices

Returns: List of aligned matrices.

8 MSNE

Source files: MSNE_feature_experiments.R, MSNE_sample_experiments.R, MSNE_run.py

Note: MSNE R scripts share get_top_features, subset_input_features, and ari_to_ground_truth with other algorithms.

8.1 parse_timev

```
parse_timev <- function(timev_str) {
  parts <- strsplit(timev_str, ":")[[1]]
  if (length(parts) == 3L) {
    as.numeric(parts[1]) * 3600 + as.numeric(parts[2]) * 60 + as.numeric(parts[3])
  } else if (length(parts) == 2L) {
    as.numeric(parts[1]) * 60 + as.numeric(parts[2])
  } else {
    as.numeric(timev_str)
  }
}
```

Description: Parses time strings in HH:MM:SS or MM:SS format to seconds.

Parameter	Type	Description
timev_str	character	Time string to parse

Returns: Numeric time in seconds.

9 NEMO

Source files: `NEMO_feature_experiments.R`, `NEMO_sample_experiments.R`

Note: NEMO scripts share `get_top_features`, `subset_input_features`, `ari_to_ground_truth`, `spectralClustering_eig`, and `align_common_samples` with other algorithms.

10 RWR-F

Source files: `RWR-F_feature_experiments.R`, `RWR-F_sample_experiments.R`

Note: RWR-F scripts share `get_top_features`, `subset_input_features`, and `ari_to_ground_truth` with other algorithms.

10.1 `parse_args` (RWR-F)

```
parse_args <- function() {
  args <- commandArgs(trailingOnly = TRUE)
  if (length(args) < 3L) stop("Usage: Rscript script.R <input_rds> <output_dir> <n_clusters>")
  list(
    input_rds = args[1],
    output_dir = args[2],
    n_clusters = as.integer(args[3])
  )
}
```

Description: Parses command-line arguments for RWR-F scripts.

Parameter	Type	Description
(none)	-	Reads from <code>commandArgs</code>

Returns: List with `input_rds`, `output_dir`, and `n_clusters`.

10.2 `specc_with_embedding`

```
specc_with_embedding <- function(affinity, K) {
  n <- nrow(affinity)
  D_inv_sqrt <- diag(1 / sqrt(rowSums(affinity) + .Machine$double.eps))
  L_sym <- D_inv_sqrt %*% affinity %*% D_inv_sqrt
  eig <- eigen(L_sym, symmetric = TRUE)
  U <- eig$vectors[, seq_len(K), drop = FALSE]
  U_norm <- U / sqrt(rowSums(U^2) + .Machine$double.eps)
  km <- kmeans(U_norm, centers = K, nstart = 25L, iter.max = 100L)
  list(labels = km$cluster, embedding = U_norm)
}
```

Description: Performs spectral clustering on an affinity matrix, returning both labels and embedding.

Parameter	Type	Description
<code>affinity</code>	matrix	Affinity/similarity matrix
<code>K</code>	integer	Number of clusters

Returns: List with cluster labels and normalized eigenvector embedding.

11 SNF

Source files: `SNF_feature_experiments.R`, `SNF_sample_experiments.R`

Note: SNF scripts share `get_top_features`, `subset_input_features`, `ari_to_ground_truth`, and `spectralClustering_eig` with other algorithms.

12 Spectrum

Source files: `Spectrum_feature_experiments.R`, `Spectrum_sample_experiments.R`

Note: Spectrum scripts share `get_top_features`, `subset_input_features`, and `ari_to_ground_truth` with other algorithms.

12.1 ref_modality

```
ref_modality <- function(input_list, prefer = "RNAseq") {
  non_null <- names(input_list)[!vapply(input_list, is.null, logical(1))]
  if (length(non_null) == 0) stop("All modalities are NULL.")
  if (prefer %in% non_null) return(prefer)
  non_null[1]
}
```

Description: Selects a reference modality from an input list, preferring a specified modality if available.

Parameter	Type	Description
<code>input_list</code>	list	Named list of modality matrices
<code>prefer</code>	character	Preferred modality name (default: "RNAseq")

Returns: Name of the selected reference modality.

12.2 safe_silhouette_avg

```
safe_silhouette_avg <- function(assignments, embeddings) {
  cl <- as.integer(assignments)
  if (length(unique(cl)) < 2L || nrow(embeddings) < 3L) return(NA_real_)
  sil <- silhouette(cl, dist = Rfast::Dist(as.matrix(embeddings), method = "euclidean"))
  summary(sil)$avg.width
}
```

Description: Computes average silhouette width with validation for edge cases.

Parameter	Type	Description
<code>assignments</code>	vector	Cluster assignments
<code>embeddings</code>	matrix	Sample embeddings for distance computation

Returns: Numeric average silhouette width, or NA for invalid inputs.

13 wMKL

Source files: `wMKL_feature_experiments.R`, `wMKL_sample_experiments.R`

Note: wMKL scripts share `get_top_features`, `subset_input_features`, `ari_to_ground_truth`, and `mad_based_weights` with other algorithms.

13.1 weighted_mutations

```
weighted_mutations <- function(features, drivers, driver_weight = 0.8, nondriver_weight = 0.2) {
  w <- ifelse(features %in% drivers, driver_weight, nondriver_weight)
  w[!is.finite(w)] <- 0
  if (sum(w) == 0) w <- rep(1, length(w))
  w <- w / sum(w)
  names(w) <- features
  w
}
```

Description: Computes normalized weights for mutation features, upweighting driver genes.

Parameter	Type	Description
<code>features</code>	character	Feature names
<code>drivers</code>	character	Driver gene names
<code>driver_weight</code>	numeric	Weight for driver features (default: 0.8)
<code>nondriver_weight</code>	numeric	Weight for non-driver features (default: 0.2)

Returns: Named numeric vector of normalized weights.

13.2 extract_eigenspace

```
extract_eigenspace <- function(affinity, K, type = 3) {
  d <- rowSums(affinity)
  d[d == 0] <- .Machine$double.eps
  # ... (Laplacian computation and eigendecomposition)
  list(labels = labels, U = U, eigDiscrete = eigDiscrete)
}
```

Description: Extracts eigenspace representation and cluster labels from an affinity matrix.

Parameter	Type	Description
<code>affinity</code>	matrix	Affinity/similarity matrix
<code>K</code>	integer	Number of clusters
<code>type</code>	integer	Laplacian type (1, 2, or 3)

Returns: List with `labels` (cluster assignments), `U` (eigenvectors), and `eigDiscrete` (discretized assignments).

13.3 safe_avg_sil

```
safe_avg_sil <- function(labels, U) {
  cl <- as.integer(labels)
  if (length(unique(cl)) < 2L || nrow(U) < 3L) return(NA_real_)
  sil <- silhouette(cl, dist = Rfast::Dist(as.matrix(U), method = "euclidean"))
  summary(sil)$avg.width
}
```

Description: Computes average silhouette width from cluster labels and eigenvector embeddings.

Parameter	Type	Description
<code>labels</code>	vector	Cluster assignments
<code>U</code>	matrix	Eigenvector embeddings

Returns: Numeric average silhouette width, or NA for invalid inputs.

14 Python Scripts

14.1 MONET (run_MONET_mainloop.py)

14.1.1 set_seeds (Python)

```
def set_seeds(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
```

Description: Sets random seeds for reproducibility in both Python's random module and NumPy.

Parameter	Type	Description
seed	int	Random seed value

14.1.2 load_correlation_matrices

```
def load_correlation_matrices(directory: str) -> dict[str, pd.DataFrame]:
    csv_files = sorted(glob.glob(os.path.join(directory, "*.csv")))
    if len(csv_files) != 5:
        raise ValueError(f"Expected 5 CSV files in {directory}, found {len(csv_files)}.")
    data: dict[str, pd.DataFrame] = {}
    for file_path in csv_files:
        omic_name = os.path.splitext(os.path.basename(file_path))[0]
        data[omic_name] = _load_one_csv(file_path)
    return data
```

Description: Loads correlation matrices from CSV files for all omics modalities.

Parameter	Type	Description
directory	str	Directory containing correlation CSV files

Returns: Dictionary mapping omic names to correlation DataFrames.

14.2 MSNE (MSNE_run.py)

14.2.1 set_seeds (MSNE Python)

```
def set_seeds(seed: int) -> None:
    np.random.seed(seed)
    random.seed(seed)
```


Description: Sets random seeds for reproducibility.

Parameter	Type	Description
seed	int	Random seed value

14.2.2 load_views_from_csv

```
def load_views_from_csv(dists_dir: str) -> list[pd.DataFrame]:
    csvs = sorted(glob.glob(os.path.join(dists_dir, "*.csv")))
    if len(csvs) != 5:
        raise ValueError(f"Expected 5 distance CSVs in {dists_dir}, found {len(csvs)}")
    views = []
    for fp in csvs:
        df = pd.read_csv(fp, index_col=0)
        views.append(df)
    return views
```

Description: Loads distance matrices from CSV files as views for MSNE.

Parameter	Type	Description
dists_dir	str	Directory containing distance CSV files

Returns: List of DataFrames representing views.

15 Post-Analysis Scripts (automated_scripts/)

These functions are used AFTER HPC benchmark runs to aggregate, visualize, and analyze results. They are not part of the algorithm execution itself.

15.1 benchmark_plotting_functions.R

Source file: Scripts/automated_scripts/benchmark_plotting_functions.R

This script provides comprehensive visualization and analysis functions for benchmark results. Key capabilities include:

- Reading and standardizing performance TSV files from various algorithm outputs
- Creating publication-ready plots for ARI, runtime, and memory metrics
- Fitting empirical time complexity models
- Computing stability metrics across replicate runs

15.1.1 theme_benchmark

```
theme_benchmark <- function(base_size = 10, legend = FALSE) {
  theme_bw(base_size = base_size) +
    theme(
      panel.grid = element_blank(),
      panel.border = element_blank(),
      axis.line.x = element_line(color = "black"),
      axis.line.y = element_line(color = "black"),
      plot.title = element_text(face = "bold", hjust = 0.5),
      axis.title = element_text(face = "bold"),
      legend.position = if (isTRUE(legend)) "right" else "none",
      plot.margin = margin(5.5, 70, 5.5, 5.5, "pt")
    )
}
```

Description: Creates a clean, publication-ready ggplot2 theme with minimal gridlines and bold titles.

Parameter	Type	Description
base_size	numeric	Base font size (default: 10)
legend	logical	If TRUE, legend on right; if FALSE, legend hidden

Returns: A ggplot2 theme object.

15.1.2 guess_benchmark_dirs

```
guess_benchmark_dirs <- function(root = "Results/Performance_benchmarks")
```

Description: Locates the feature and sample perturbation directories within a benchmark results root. Handles common typos in directory names.

Parameter	Type	Description
root	character	Path to benchmark results root directory

Returns: Named list with `feature_dir` and `sample_dir` paths.

15.1.3 read_mode_perf

```
read_mode_perf <- function(mode = c("feature", "sample"),
                           root = "Results/Performance_benchmarks",
                           method_categories_map = method_categories)
```

Description: Discovers, reads, and standardizes all performance TSV files for either feature or sample perturbation experiments across all algorithms.

Parameter	Type	Description
mode	character	Either “feature” or “sample” perturbation mode
root	character	Path to benchmark results root directory
method_categories_map	named vector	Mapping of algorithms to categories

Returns: Aggregated data.table with standardized metrics per algorithm/subset combination.

15.1.4 plot_metric_lines

```
plot_metric_lines <- function(dt, mode, metric, x_axis, title, ylab, category_colors_map, legend)
```

Description: Generates a publication-ready line plot showing how a metric varies with subset size across algorithms.

Parameter	Type	Description
dt	data.table	Aggregated data from <code>read_mode_perf</code>
mode	character	Either “feature” or “sample”
metric	character	One of “ARI_to_Ground_Truth”, “Peak_MiB”, or “Time_s”

Parameter	Type	Description
x_axis	character	Either “percent” or “absolute”
title	character	Plot title
ylab	character	Y-axis label
category_colors_map	named vector	Category colors
legend	logical	Whether to show legend

Returns: A ggplot object.

15.1.5 save_plot_png_pdf

```
save_plot_png_pdf <- function(p, title, out_dir, dpi = 700, width_px = 3840, height_px = 2160)
```

Description: Saves a ggplot object to both PNG and PDF formats with consistent naming and high resolution suitable for publication.

Parameter	Type	Description
p	ggplot	Plot object to save
title	character	Title used to generate filename
out_dir	character	Output directory path
dpi	numeric	Dots per inch (default: 700)
width_px	numeric	Width in pixels (default: 3840 for 4K)
height_px	numeric	Height in pixels (default: 2160 for 4K)

Returns: Invisible NULL.

15.1.6 fit_time_scaling_models

```
fit_time_scaling_models <- function(root = "Results/Performance_benchmarks",
                                     method_categories_map = method_categories,
                                     min_points = 4)
```

Description: Fits power-law models ($T \sim \text{size}^k$) to runtime data for both feature and sample perturbation modes across all algorithms. Returns estimated complexity in Big-O notation.

Parameter	Type	Description
root	character	Path to benchmark results root directory
method_categories_map	named vector	Mapping of algorithms to categories
min_points	integer	Minimum data points for fitting (default: 4)

Returns: Data.table with columns: Mode, Algorithm, Category, Predictor, Exponent, R2, O_notation.

15.1.7 compute_sample_stability_90pct

```
compute_sample_stability_90pct <- function(root = "Results/Performance_benchmarks",
  out_dir = "Results/Comparisons",
  pct = 90, min_common_sample = 20L,
  save = TRUE, dpi = 700,
  width_px = 3840, height_px = 2160)
```

Description: Analyzes replicate stability by computing pairwise ARI between clusterings at a target sample perturbation percentage. Creates violin plots and exports summary tables.

Parameter	Type	Description
root	character	Path to benchmark results root directory
out_dir	character	Output directory for files
pct	numeric	Target perturbation percentage (default: 90)
min_common_sample	integer	Minimum common samples for ARI (default: 20)
save	logical	Whether to save plots and tables

Returns: List with pairs, summary, skipped, and plot objects.

15.1.8 make_benchmark_plots

```
make_benchmark_plots <- function(root = "Results/Performance_benchmarks",
  out_dir = "Results/Comparisons",
  x_axis = c("percent", "absolute"),
  save = TRUE, dpi = 700,
  width_px = 3840, height_px = 2160,
  method_categories_map = method_categories,
  category_colors_map = category_colors,
  legend = FALSE)
```

Description: Main entry point for benchmark visualization. Reads performance data for both feature and sample perturbation experiments, generates line plots for ARI, runtime, and memory metrics, and optionally saves to disk.

Parameter	Type	Description
root	character	Path to benchmark results root directory
out_dir	character	Output directory for files
x_axis	character	Either “percent” or “absolute”
save	logical	Whether to save plots (default: TRUE)
method_categories_map	named vector	Algorithm to category mapping
category_colors_map	named vector	Category color palette
legend	logical	Whether to show legend (default: FALSE)

Returns: Invisible list of named plot objects with titles.

16 Alphabetical Index

Function	Source File(s)
align_by_samples	COCA
align_common_samples	MONET, NEMO
align_modalities	iClusterBayes, LRAcluster
ari_to_ground_truth	ab-SNF, ANF, CIMLR, COCA, iClusterBayes, KLIC, LRAcluster, MDICC, NEMO, SNF, Spectrum, wMKL
coca_distances_for	COCA
compute_sample_stability_90pct	benchmark_plotting_functions.R (Post-Analysis)
ensure_bioc_pkg	iClusterBayes
ensure_cran_pkg	iClusterBayes
ensure_mosek_env	KLIC
ensure_projsplx	MDICC
ensure_python	MDICC
ensure_rmosek	KLIC
extract_eigenspace	wMKL
extract_m3c_clusters	CIMLR, LRAcluster
extract_m3c_scores	CIMLR
fallback_MDICClabel	MDICC
fallback_testaff	MDICC
fit_time_scaling_models	benchmark_plotting_functions.R (Post-Analysis)
get_intl	ab-SNF, COCA
get_mdicc_field	MDICC
get_top_features	ab-SNF, ANF, CIMLR, COCA, iClusterBayes, KLIC, LRAcluster, MDICC, MONET, MSNE, NEMO, RWR-F, SNF, Spectrum, wMKL
guess_benchmark_dirs	benchmark_plotting_functions.R (Post-Analysis)
load_correlation_matrices	MONET (Python)
load_views_from_csv	MSNE (Python)
mad_based_weights	ab-SNF, wMKL
make_benchmark_plots	benchmark_plotting_functions.R (Post-Analysis)
parse_args	RWR-F
parse_timev	MSNE
plot_metric_lines	benchmark_plotting_functions.R (Post-Analysis)
rank_mad_ids	Create_feature_ranks.R
read_mode_perf	benchmark_plotting_functions.R (Post-Analysis)
ref_modality	Spectrum
safe_avg_sil	wMKL
safe_silhouette_avg	Spectrum
save_plot_png_pdf	benchmark_plotting_functions.R (Post-Analysis)
same_conda_prefix	MDICC
set_seeds	MONET (Python), MSNE (Python)
specc_with_embedding	RWR-F
spectralClustering_eig	ab-SNF, ANF, NEMO, SNF
strat_sample_ids	Create_sample_subsets.R
subset_input_cols	ab-SNF, CIMLR, COCA, iClusterBayes, KLIC, LRAcluster, MDICC
subset_input_features	ab-SNF, ANF, CIMLR, COCA, iClusterBayes, KLIC, LRAcluster, MDICC, MONET, MSNE, NEMO, RWR-F, SNF, Spectrum, wMKL
theme_benchmark	benchmark_plotting_functions.R (Post-Analysis)

Function	Source File(s)
to_sample_feature	KLIC, MDICC
to_samples_x_features	COCA
types_by_mod	LRACluster
weighted_euclidean_dist	ab-SNF
weighted_hamming_fraction_fast	ab-SNF
weighted_mutations	wMKL
with_install_lock	iClusterBayes
